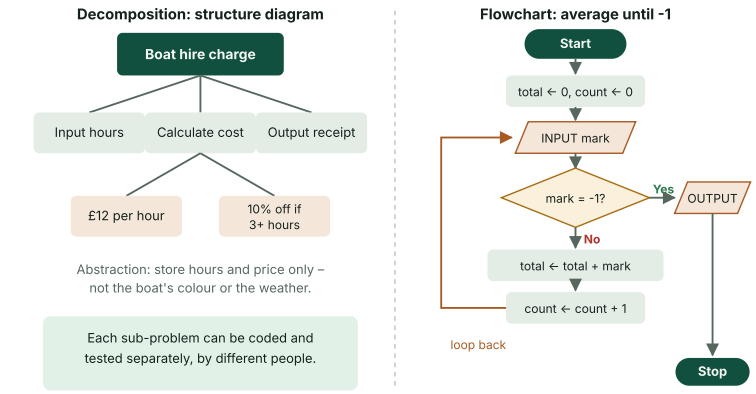
Programs, algorithms and their design

AP16 FACT FILE

Specification: students should be able to

- Describe the purpose of a computer program
- Explain the term algorithm and the properties a good algorithm must have
- Explain decomposition and abstraction as techniques for solving problems
- Describe how algorithms can be represented using pseudocode and flowcharts
- Use trace tables and dry runs to test an algorithm and find logic errors

Key words: computer program, application program, algorithm, finiteness, unambiguous, decomposition, abstraction, pseudocode, structured English, flowchart, input-process-output, sequence, selection, iteration, sentinel value, trace table, dry run, logic error



Decomposition of a boat-hire problem into a structure diagram, beside a flowchart of a sentinel-controlled averaging loop

From problem to program

A computer program is a set of instructions that a computer follows to carry out a task. Programs are software, which distinguishes them from the hardware they run on. Some software, such as the operating system, provides an interface so that people can use and control the computer; application programs, such as word processors, spreadsheets or a sports club's booking app, are designed to carry out a particular job for the user. Before any code is written, the programmer must work out exactly how the task will be done. That plan is an algorithm: a finite, ordered set of precise and unambiguous steps that solves a problem. Algorithms are not unique to computing – a recipe or a set of directions is an algorithm – but a computer is far less forgiving than a person, because it cannot use common sense to fill gaps or resolve ambiguity. Designing the algorithm first, using decomposition and abstraction, and expressing it in pseudocode or as a flowchart, allows the logic to be checked with a dry run before time is spent coding. Logic errors found at this stage are far cheaper and quicker to fix than errors found after the program has been written and released.

What makes a good algorithm

A useful algorithm has several properties. Its steps are in a specific order, and that order matters: outputting a total before it has been calculated gives the wrong answer. Each step must be precise, so 'wait a few minutes' is replaced by 'wait 5 minutes', and unambiguous, so that it can be interpreted in only one way. An algorithm must be finite: it must terminate after a limited number of steps, which means every loop needs a condition that will eventually be met. An instruction such as 'repeat until the cake rises' might never end. An algorithm takes zero or more inputs, may process them, and produces at least one output that is related to those inputs. It must be correct and effective, working for every valid case and dealing sensibly with invalid input, for example by rejecting a negative quantity with a clear message rather than crashing. Finally, it should be efficient, avoiding unnecessary steps or slow methods when a quicker one would do the job equally well. When asked to evaluate an algorithm, test it against each of these properties in turn.

Decomposition and abstraction

Computational thinking starts with decomposition: splitting a large problem into smaller, manageable sub-problems that can be designed, coded and tested separately, often by different programmers working at the same time. A structure diagram shows this top-down breakdown, with the whole problem at the top and progressively smaller tasks beneath it. Each sub-problem becomes a module, such as a procedure or function, which can be reused elsewhere and is easier to maintain. Abstraction removes detail that is not needed for the problem, producing a simpler model. For example, a cinema booking system stores seat numbers, prices and screening times but not the colour of the seats, and a bus route map shows stops and connections rather than every bend in the road. Abstraction reduces the amount of data to be stored and processed and lets designers concentrate on the parts that matter. Used together, the two techniques make problems easier to understand, reduce development time, allow work to be shared across a team and lead to modular code that is simpler to test and debug.

Representing algorithms

An algorithm can be written in plain English, which has the advantage that a client with no programming experience can read it and confirm that it describes the right process. Algorithms that are going to become programs are usually written in structured English or pseudocode instead, which use keywords and indentation that resemble program code. These are more helpful to the programmer, though slightly harder for a non-specialist to follow. Pseudocode has no single official syntax; what matters is that it is consistent and unambiguous and shows sequence, selection and iteration clearly. A flowchart represents the same logic as a diagram using standard symbols: a terminator for START and END, a rectangle for a process, a parallelogram for input or output, a diamond for a decision with two labelled exits, and arrows for the flow of control. A loop appears as an arrow returning to an earlier decision. Flowcharts are visual and easy to follow for small problems but become unwieldy for large ones, whereas pseudocode is quicker to write and translates more directly into code. Both are language-independent.

Building blocks and loops

Every algorithm is built from three constructs. Sequence means steps are carried out one after another in order. Selection means a decision is made and different steps are followed depending on whether a condition is true or false, as in IF...THEN...ELSE. Iteration means a set of steps is repeated. A count-controlled loop, such as FOR day ← 1 TO 4, is used when the number of repetitions is known before the loop starts. A condition-controlled loop, such as WHILE or REPEAT...UNTIL, is used when it is not, for example reading exam marks until the user enters a sentinel value of -1. A WHILE loop tests its condition at the start, so its body may run zero times; a REPEAT...UNTIL loop tests at the end, so its body always runs at least once. Variables used to accumulate a total or count occurrences must be initialised, usually to zero, before the loop begins, and must not be reset inside it. In a flowchart, both kinds of loop are drawn with a decision diamond and a flow line back to an earlier point.

Testing a design with a trace table

A dry run tests an algorithm by working through it by hand, one step at a time, without a computer. The results are recorded in a trace table, which has a column for each variable, often a column for any condition being tested, and a column for output. A new row is added each time values change, so every pass of a loop can be seen. Because the dry run follows the algorithm exactly as it is written, rather than as the designer intended it to work, it exposes logic errors: a total that is never initialised, a counter reset inside a loop, a loop that runs one time too many or too few because < has been used instead of <=, or a missing input that causes an infinite loop. Logic errors do not stop a program running, so they are not caught by the compiler; only careful testing reveals them. Good test data includes normal values, boundary values at the edge of a condition, such as exactly 10 mm of rain when the test is greater than 10, and erroneous values that should be rejected.

T Key terms from the mark schemes

Computer program
A set of instructions that a computer executes in order to perform a specific task.
`Credited: software that carries out a task`

Application program
Software designed to perform a specific task for the user, such as a word processor or spreadsheet.
`Credited: application software`

Algorithm
A finite, ordered sequence of precise, unambiguous steps designed to solve a problem or complete a task.
Credited: a step-by-step set of instructions to solve a problem

Finiteness
The property that an algorithm must terminate after a limited number of steps.
Credited: the algorithm must end

Decomposition
Breaking a complex problem into smaller sub-problems that can be solved independently.
Credited: top-down design

Abstraction
Removing unnecessary detail from a problem so that only the relevant details remain.

Pseudocode
A structured, language-independent, English-like way of describing an algorithm that resembles program code.
Credited: structured English

Flowchart
A diagram that represents an algorithm using standard symbols joined by flow lines showing the order of steps.

Trace table
A table used during a dry run to record the value of each variable and any output at each step of an algorithm.

Dry run
Working through an algorithm or program by hand, step by step, to check its logic.
Credited: desk check

Sentinel value
A special input value, such as -1, that signals the end of the data and stops a condition-controlled loop.
Credited: rogue value

Logic error
An error in the design of an algorithm that causes incorrect output although the program still runs.

C C# example

The rainfall algorithm in C#

```
int total = 0, wetDays = 0;
for (int day = 1; day <= 4; day++)
{
    Console.Write($"Rainfall for day {day} (mm): ");
    int rain = int.Parse(Console.ReadLine());
    total += rain;
    if (rain > 10)
    {
        wetDays++;
    }
}
Console.WriteLine($"Total rainfall: {total} mm");
Console.WriteLine($"Wet days: {wetDays}");
```

Exam focus

Short questions ask you to state or define: an algorithm is a sequence of precise, unambiguous steps to solve a problem, and a fully credited answer usually includes the idea of order and of solving a problem. For 'describe the purpose of a program' say it is a set of instructions that the computer carries out to perform a specific task. Properties questions usually award one mark per property named and one for an explanation or example, so pair each property with how it applies to the scenario given. When asked to write an algorithm, examiners credit logic rather than exact syntax: initialising totals and counters, a correctly controlled loop, inputs inside the loop, the right condition in the decision and an output after the loop. Flowcharts must use the correct symbols and label both exits of every decision. In trace table questions, follow the algorithm exactly as written, complete every row and do not skip a pass of the loop. Common mistakes include confusing decomposition with abstraction, forgetting the final output, using one exit from a decision and placing the output inside the loop so it prints every time.

? Check yourself

1. A Year 8 pupil says that a program and the computer it runs on are 'the same thing'. Describe the purpose of a computer program, explaining why it is different from the hardware. [2]
2. Before coding a lunch-ordering app for a Strabane primary school, a developer writes an algorithm. Explain what is meant by an algorithm. [2]
3. An algorithm is written to guide visitors around Castle Coole. Explain three properties that the algorithm must have to be useful. [6]

Answers: 1. A set of instructions (1) / executed by the computer to carry out a specific task or solve a problem; it is software, separate from the physical hardware that runs it (1) 2. A sequence of (step-by-step) instructions (1) / that are precise/unambiguous and in a specific order, designed to solve a problem/complete a task (1) 3. Any three properties, each identified (1) and explained in context (1), e.g.: steps in a specific order (1) so visitors see rooms in the correct route (1) / each step precise (1) e.g. 'walk 20 metres to the staircase' not 'walk a bit' (1) / unambiguous (1) e.g. naming which door when there are several (1) / finite (1) the tour must end, e.g. at the exit (1) / has inputs and at least one output (1) e.g. visitor's start point in, route out (1) / correct and effective (1) deals with a closed room by giving an alternative (1) / efficient (1) no unnecessary doubling back (1)